

Avionics & Embedded
Systems Engineering

ATtiny85 Hardware Login Key

Two-Piece Password



System Requirements Specification

Authors:	Paschalis M.
Group:	None
Approval Date:	21/05/2025
Submission Date:	23/06/2025



Avionics & Embedded
Systems Engineering

ATtiny85 Hardware Login Key

Two-Piece Password



System Requirements Specification

Authors:	Paschalis M.
Group:	None
Approval Date:	21/05/2025
Submission Date:	23/06/2025

SYSTEM REQUIREMENTS SPECIFICATION

for

ATtiny85 Hardware Login Key

Version 1.0 approved

Prepared by *Paschalis Moschogiannis*

Avionics & Embedded
Systems Engineering

June 23, 2025

Revision History

Author	Date	Reason for Change	Version
Paschalis M.	23-06-2025	Initial draft of the specification.	1.0

Team Members

Name	Role	Email
Paschalis M.	Project Lead & Developer	

Contents

Revision History	i
Team Members	i
1 Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms, Abbreviations	2
1.4 References	2
2 Overall Description	3
2.1 Product Perspective	3
2.2 Product Features	3
3 System Features	4
3.1 Hardware Requirements	4
3.1.1 Core Circuit	4
3.1.2 Variant-Specific Input	4
3.1.3 PCB Schematics	4
3.2 Microcontroller Fuse Configuration	6
3.2.1 Required Fuse Settings for This Project	6
3.2.2 AVRDUDE Commands	7
3.3 Software Features	8
3.4 Use-Case Diagram	8
4 Non-Functional Requirements	9
4.1 Performance	9
4.2 Security	9
4.3 Reliability	9
5 Risk Analysis	10
6 Future Enhancements	11
Firmware Snippets	12

1 Introduction

1.1 Purpose

This document specifies the software and hardware requirements for the **ATtiny85 Hardware Login Key**. The device is a small, low-cost hardware key designed to enhance login security by automating the entry of a complex portion of a user's password via USB HID keyboard emulation.

The primary goal is to mitigate risks associated with manual password entry—such as shoulder surfing and keystroke observation—by implementing a *two-part password* methodology. The user manually types a memorable prefix; the hardware key types a complex, pre-programmed suffix upon a button press or capacitive touch event.

Two firmware variants are provided, sharing the same security model and USB HID stack:

- **Tactile Switch Variant:** Physical push-button on PB1 (active-low, internal pull-up resistor enabled in firmware).
- **Touch Pad Variant:** TTP223-BA6 capacitive touch IC on PB1 (active-high, no pull-up; dedicated IC provides hardware debounce and reliable touch detection).

1.2 Scope

This SRS covers the ATtiny85-based Hardware Login Key in all four PCB variants (Through-Hole DIP, SMD SOIC, SMD QFN, SMD QFN with TTP223 touch). It does not cover future platform migrations to 32-bit microcontrollers (STM32, RP2350), which are addressed separately.

1.3 Definitions, Acronyms, Abbreviations

Term	Definition
ATtiny85	Low-power, 8-bit AVR microcontroller (Atmel/Microchip). 8 KB Flash, 512 B SRAM, 512 B EEPROM.
V-USB	Firmware-only USB 1.1 low-speed driver for AVR microcontrollers (Objective Development).
HID	Human Interface Device. USB device class for keyboards, mice, and similar peripherals.
ISP	In-System Programming. Uploads firmware to the MCU via a 6-pin SPI interface (MOSI, MISO, SCK, RESET, VCC, GND).
AVRDUDE	Open-source tool for reading and writing AVR microcontroller flash, EEPROM, and fuse bytes via a programmer.
Fuse Byte	Non-volatile configuration bytes in the AVR that control hardware behaviour (clock source, BOD voltage, debug interface). Written via ISP; not modifiable by firmware at runtime.
lfuse	Low fuse byte. Configures clock source and startup timing (e.g., external crystal vs. internal oscillator).
hfuse	High fuse byte. Configures Brown-Out Detector level, SPI programming enable, RESET pin function, and watchdog.
BOD	Brown-Out Detector. Hardware circuit that holds the MCU in reset if VCC drops below a configured threshold voltage, preventing undefined behaviour during power instability.
EEPROM	Electrically Erasable Programmable Read-Only Memory. Persistent storage writable by firmware at runtime. Not used in this project (suffix is stored in Flash).
Suffix	The complex second part of the password, stored in ATtiny85 Flash and typed by the device.
Prefix	The simple first part of the password, memorised and typed manually by the user.
TTP223-BA6	Capacitive touch IC providing a digital high/low output upon pad contact. Used in the QFN touch variant.

1.4 References

- Atmel/Microchip, *ATtiny25/45/85 Datasheet* (doc. no. Atmel-2586), <https://ww1.microchip.com/downloads/en/DeviceDoc/atmel-2586-avr-8-bit-microcontroller-attiny25-attiny45-attiny85-datasheet.pdf>
- Objective Development, *V-USB: A Firmware-Only USB Driver for AVR Microcontrollers*, <https://www.obdev.at/products/vusb/index.html>
- Project repository: <https://github.com/Paschalis/Hardware-Login-Key>
- AVR Fuse Calculator: <https://www.engbedded.com/fusecalc/>

2 Overall Description

2.1 Product Perspective

The ATtiny85 Hardware Login Key is a standalone, bus-powered USB peripheral. It is recognised by host operating systems (Windows, macOS, Linux) as a generic USB HID Keyboard—no drivers are required. All functionality is self-contained in the ATtiny85 firmware. The device does not depend on network or cloud services and has no wireless interface.

2.2 Product Features

- **USB HID Keyboard Emulation:** Presents as a standard USB keyboard to any host OS via the V-USB library.
- **Two-Part Security Model:** Stores only the complex password suffix on-device. The prefix exists only in the user's memory. Physical possession of the key alone is insufficient to authenticate.
- **Tactile Switch Variant:** Button press on PB1 triggers suffix injection (active-low, 20 ms debounce in firmware).
- **Touch Pad Variant:** Capacitive touch via TTP223-BA6 on PB1 triggers suffix injection (active-high). The IC provides hardware debounce; the firmware waits for the signal to go low before accepting a new trigger.
- **Four PCB Variants:** Through-Hole DIP (prototyping), SMD SOIC-8 (compact), SMD QFN-20 with button, SMD QFN-20 with TTP223 touch (4-layer PCB for optimal signal integrity).
- **3D-Printed Enclosure:** Press-fit PETG/PLA enclosure; no fasteners required.
- **Firmware-Defined Payload:** Keystroke sequence is hard-coded in Flash. No external configuration file exists.

3 System Features

3.1 Hardware Requirements

3.1.1 Core Circuit

All four PCB variants share the following circuit topology:

- **Microcontroller:** Atmel ATtiny85 (DIP-8, SOIC-8, or QFN-20 package depending on variant).
- **Clock Source:** 16 MHz external crystal with two 22 pF loading capacitors. Mandatory for V-USB: the USB full-speed protocol requires ± 1500 ppm frequency accuracy; the internal oscillator ($\pm 10\%$) is insufficient.
- **USB Interface:**
 - 68 Ω series resistors on D+ and D– (impedance matching).
 - 1.5 k Ω pull-up on D– (signals a USB low-speed device to the host).
 - 3.6 V Zener diodes on D+ and D– (clamp to protect ATtiny85 pins from USB 5 V transients; the pins are 5 V-tolerant but the Zeners prevent glitches during enumeration).
- **Power Decoupling:** 100 nF ceramic capacitor between VCC and GND, placed close to the MCU power pins.
- **RESET Pull-up:** 10 k Ω resistor on the RESET pin prevents spurious resets from noise on the line.

3.1.2 Variant-Specific Input

- **Tactile Switch Variants (DIP, SOIC, QFN):** One momentary push-button connected between PB1 and GND. The firmware enables the internal pull-up on PB1 (approximately 50 k Ω). A press drives PB1 low.
- **Touch Pad Variant (QFN + TTP223):** TTP223-BA6 IC with output connected to PB1. The IC drives PB1 high when touch is detected and low otherwise. The firmware **does not** enable the internal pull-up on PB1 for this variant. Additional capacitor C5 (30 pF) sets the TTP223 sensitivity.

3.1.3 PCB Schematics

Figure 3.1 shows the Through-Hole DIP schematic used as the reference topology for all variants. Figure 3.2 shows the QFN + TTP223 touch variant with the capacitive input circuit.

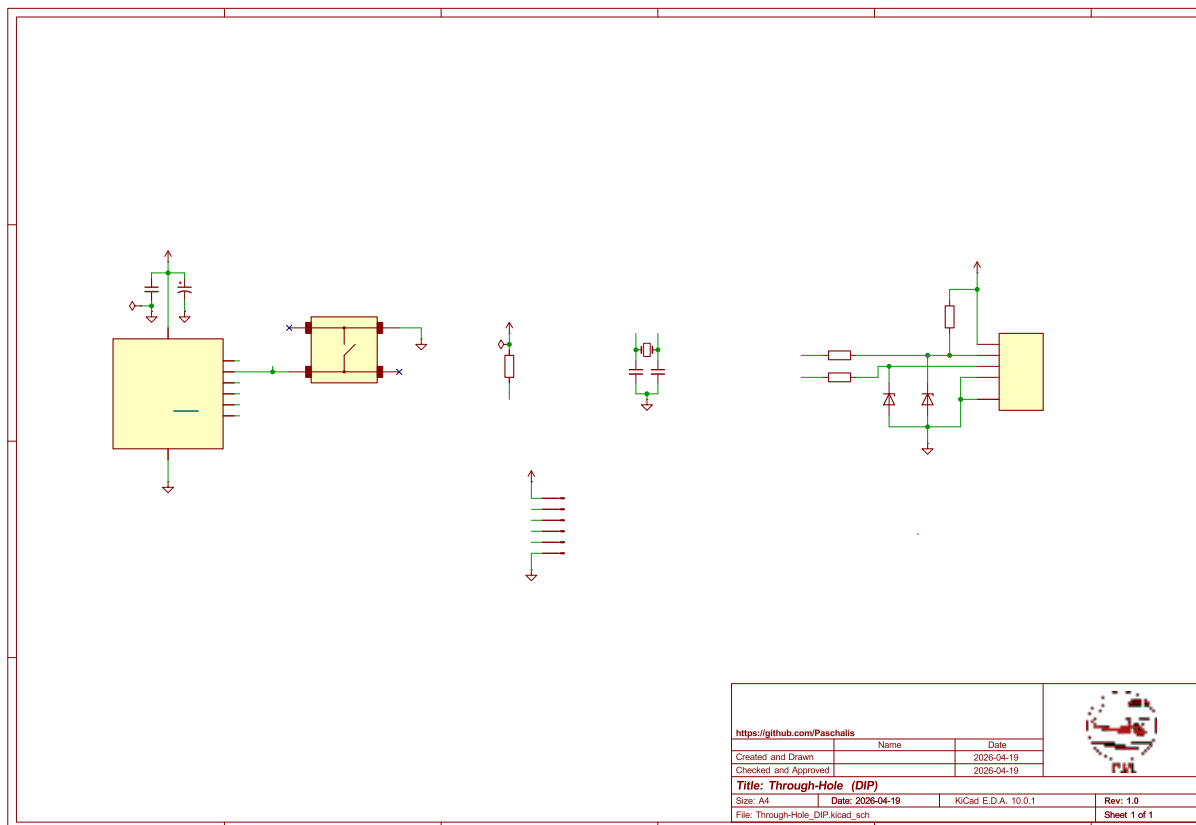


Figure 3.1: Hardware schematic — Through-Hole (DIP) variant (KiCad). All tactile switch variants share this topology.

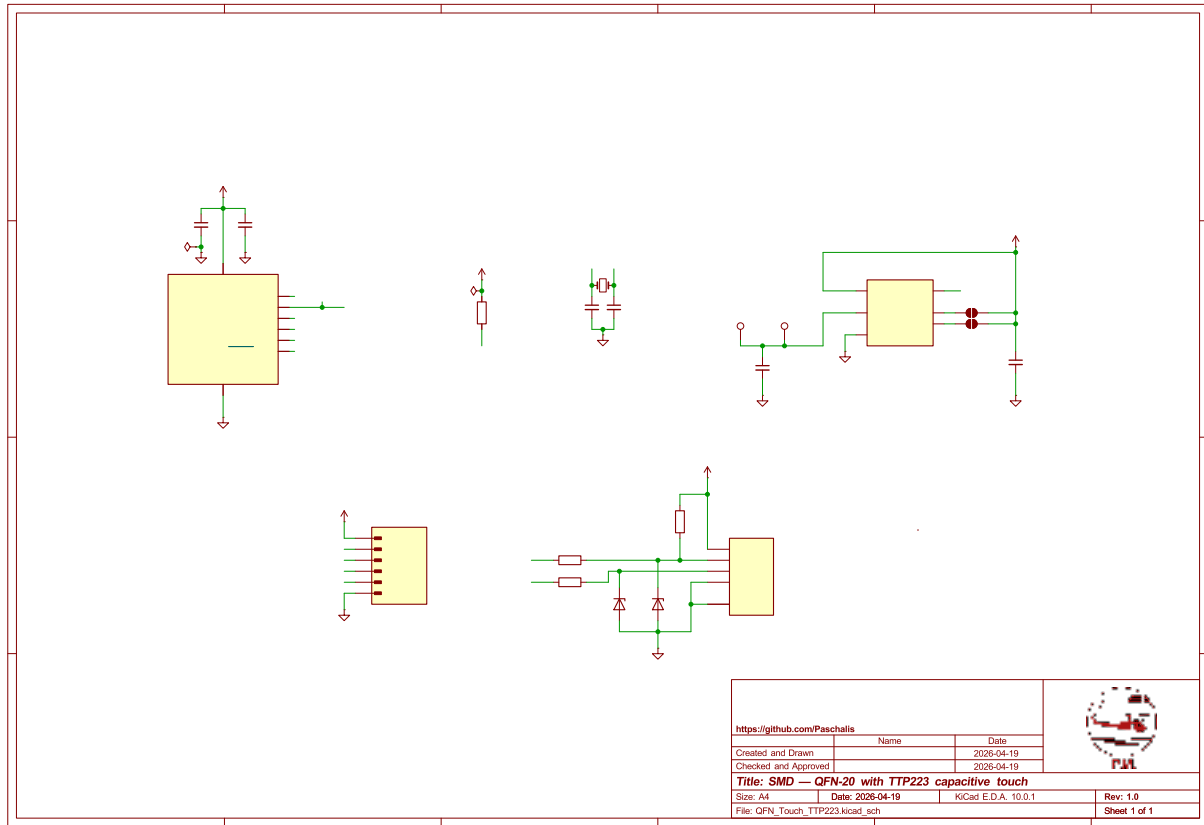


Figure 3.2: Hardware schematic — QFN + TTP223 touch variant (KiCad). Differs from the DIP variant by the TTP223-BA6 input IC, sensitivity capacitor C5, and the 4-layer PCB stackup.

3.2 Microcontroller Fuse Configuration

AVR fuse bytes are non-volatile configuration cells separate from Flash and EEPROM. They control chip-level hardware behaviour and are read by the ATtiny85 during reset, before any firmware executes. They can only be written via an external ISP programmer (AVRDUDE); firmware cannot modify them at runtime.

AVR fuse bit polarity convention: a bit value of 0 means the fuse is *programmed* (feature active), and 1 means *unprogrammed* (feature inactive). This is the inverse of what is intuitively expected and is a common source of errors.

3.2.1 Required Fuse Settings for This Project

Byte	Bit Name	Value	Effect in this project
8*lfuse	CKDIV8	1	Clock NOT divided by 8 — full 16 MHz used
	CKOUT	1	Clock output on PB4 disabled
	SUT1	1	2*Maximum start-up delay (crystal stabilisation before code runs)
	SUT0	1	
	CKSEL3	1	4*External crystal oscillator, 8–16 MHz, full swing (CKSEL = 1111)
	CKSEL2	1	
	CKSEL1	1	
	CKSEL0	1	
8*hfuse	RSTDISBL	1	RESET pin active (ISP works). Setting to 0 disables RESET and locks out ISP — recovery requires 12 V HVSP.
	DWEN	1	debugWIRE disabled
	SPIEN	0	SPI programming enabled (AVRDUDE works). Must never be set to 1 while programming via SPI.
	WDTON	1	Watchdog timer not forced always-on
	EESAVE	1	EEPROM erased on chip erase (not used in this project)
	BODLEVEL2	1	3*BOD threshold = 2.7 V (BODLEVEL = 101). Prevents undefined execution during USB hot-plug VCC dips. Default 0xDF has BOD disabled.
	BODLEVEL1	0	
	BODLEVEL0	1	

3.2.2 AVRDUDE Commands

Set fuses for 16 MHz external crystal (run once per fresh chip):

```
1 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
2 -U lfuse:w:0xFF:m -U hfuse:w:0xDD:m
```

- `-c avrisp`: Arduino running the ArduinoISP sketch as programmer.
- `-p attiny85`: Target chip model (determines memory map and fuse layout).
- `-P /dev/ttyACM0`: Serial port of the Arduino (replace with `/dev/cu.*` on macOS or `COMx` on Windows).
- `-b 19200`: Baud rate matching the ArduinoISP sketch.
- `-U lfuse:w:0xFF:m`: Write 0xFF to low fuse (external crystal, full swing, max start-up delay).
- `-U hfuse:w:0xDD:m`: Write 0xDD to high fuse (SPI on, RESET on, BOD at 2.7 V).

Revert to internal 8 MHz oscillator (no crystal needed):

```
1 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
2 -U lfuse:w:0xE2:m -U hfuse:w:0xDF:m
```

Fuse Byte	Value (hex)	Meaning
lfuse	0xFF	External crystal, 8–16 MHz, full swing, max start-up delay
hfuse	0xDD	SPI on, RESET active, BOD at 2.7 V
lfuse	0xE2	Internal 8 MHz oscillator, no clock divide
hfuse	0xDF	SPI on, RESET active, BOD disabled (factory default)

Why hfuse=0xDD instead of 0xDF? The only difference is BODLEVEL1 (bit 1). 0xDD enables the Brown-Out Detector at 2.7 V. USB power can dip briefly during host enumeration and on plug events. Without BOD, the ATtiny85 may begin executing at an indeterminate voltage, causing V-USB enumeration failures or corrupt behaviour. The 2.7 V threshold ensures a clean reset-and-restart whenever VCC is unstable.

The danger of RSTDISBL. If RSTDISBL (hfuse bit 7) is accidentally programmed to 0, the RESET pin becomes a GPIO and ISP access is permanently lost. The only recovery method is High-Voltage Serial Programming (HVSP), which requires applying 12 V to pin 1. Always verify your fuse hex values at <https://www.engbedded.com/fusecalc/> before running any write command.

3.3 Software Features

- **Firmware Language:** C (avr-gcc toolchain).
- **USB Stack:** V-USB library (GPLv2). Implements USB 1.1 low-speed HID keyboard in firmware; no hardware USB peripheral is used.
- **Keystroke Payload:** A `const char` array containing the password suffix is stored in ATtiny85 Flash as a C string literal. It is not in EEPROM.
- **Input Handling:**
 - Tactile variant: polls PB1 for LOW; 20 ms software debounce; waits for release before re-arming.
 - Touch variant: polls PB1 for HIGH; 20 ms debounce; waits for LOW (touch released) before re-arming.
- **HID Report Descriptor:** Standard 63-byte boot-protocol keyboard descriptor is included in Flash (`PROGMEM const char usbHidReportDescriptor[63]`).
- **Firmware Size:** Approximately 1.86 KB Flash (22% of 8 KB) for the tactile variant. Confirmed by CI via `avr-size --format=avr --mcu=attiny85 main.elf`.

3.4 Use-Case Diagram

Actor	Use Case
User	Types password prefix in the login field, then presses the button (or touches the pad) to have the device inject the suffix followed by Enter.
Developer	Connects ISP programmer to the 6-pin header to flash new firmware with a different password suffix. Runs fuse programming once per new chip.
Attacker (physical)	Gains possession of the device and reads Flash via ISP to extract the plaintext suffix. Without the user-memorised prefix, authentication still fails.

4 Non-Functional Requirements

4.1 Performance

- **Enumeration Time:** The device shall be recognised as a USB HID Keyboard within 2 seconds of connection.
- **Response Latency:** Keystroke injection shall begin within 50 ms of a confirmed button press or touch event.
- **Inter-Keystroke Delay:** Each keystroke shall be followed by a 50 ms delay to ensure compatibility with slow host input stacks and web-based login forms.

4.2 Security

- The device shall not store the complete password—only the suffix.
- The suffix is stored as plaintext in ATtiny85 Flash. **This is an accepted architectural limitation.** An attacker with physical access and an ISP programmer can extract the suffix with AVRDUDE in under one minute. The security model assumes the prefix remains undisclosed; physical theft of the device alone does not grant authentication.
- No configuration file, no serial/Bluetooth interface, and no wireless attack surface exists.
- Physical access to the device and ISP programmer is required to reprogram the suffix.
- The ATtiny85 has no hardware security primitives (no secure boot, no OTP key storage, no encrypted Flash). Deployments requiring resistance to physical flash extraction must use a more capable platform (e.g. RP2350 with OTP and encrypted boot, or STM32 with readout protection).

4.3 Reliability

- The device shall operate reliably on any standard USB 2.0 or USB 3.0 port.
- The external 16 MHz crystal is mandatory for stable V-USB timing across temperature and host controller variation.
- The BOD at 2.7 V (`hfuse=0xDD`) shall prevent undefined firmware execution during USB VCC instability.
- The TTP223 variant shall be resistant to false triggers from nearby conductive objects; sensitivity is set by capacitor C5 (30 pF default).

5 Risk Analysis

Physical Theft An attacker obtains the device.

Impact: Attacker can attempt to extract the suffix.

Mitigation: Only the suffix is on-device. Without the prefix (user memory), the suffix is not sufficient to authenticate on any system that requires the full password. Risk is accepted.

Flash Extraction via ISP An attacker with an Arduino and AVRDUDE reads the Flash.

Impact: Suffix is fully exposed within ~1 minute.

Mitigation: Accepted risk for this platform. The ATtiny85 has no hardware readout protection. Mitigated at the system level by the two-part model. Future migration to RP2350 (OTP + encrypted boot) or STM32 (RDP Level 2) will close this gap.

Incorrect Fuse Programming Developer sets RSTDISBL=0 or SPIEN=1 in hfuse.

Impact: ISP access permanently lost; chip requires HVSP (12 V on pin 1) to recover, which most developers cannot perform without dedicated hardware.

Mitigation: Documentation includes fuse verification via <https://www.engbedded.com/fusecalc/> before any write. The full fuse bit table is provided in Section 3.2.

USB Power Instability VCC dips during plug events or on poor-quality USB hubs.

Impact: ATtiny85 executes undefined instructions; V-USB enumeration fails or corrupts.

Mitigation: BOD at 2.7 V (hfuse=0xDD) forces a clean reset on VCC drop. 100 nF decoupling capacitor on VCC.

Keystroke Interception on Host Host OS keylogger or malicious process captures USB HID events.

Impact: Suffix captured in transit on a compromised host.

Mitigation: Out of scope for this hardware device. Users should not use any hardware key on an untrusted host.

Clock Failure External crystal not fitted, damaged, or loading caps wrong value.

Impact: Chip operates from no clock (freezes) or at incorrect frequency; V-USB fails to enumerate.

Mitigation: Documentation mandates crystal and 22 pF caps. If lfuse is set to 0xFF without a crystal present, the chip will appear unresponsive but is not damaged; fitting the crystal restores operation.

6 Future Enhancements

1. **Status LED:** Add an indicator LED for visual feedback (solid = ready, blinking = typing in progress).
2. **Multi-Password Support:** Use the ATtiny85 EEPROM (512 B) with an index selected by number of button presses before the long-press trigger.
3. **Platform Migration — STM32:**
 - Native USB Full Speed (no V-USB required).
 - OLED display (SSD1306) for multi-entry selection.
 - Rotary encoder for PIN-entry before payload unlock.
 - STM32 Flash Read-Out Protection (RDP) Level 1/2 to partially mitigate ISP extraction.
 - On-device AES encryption of stored secrets.
4. **Platform Migration — RP2350:**
 - OTP memory for immutable key storage.
 - Encrypted boot (AES-128) for firmware in external Flash.
 - Secure boot with ECDSA signature verification against OTP key.
 - Debug lockdown via OTP.
 - Hardware glitch detection (power and clock).
 - TrustZone-M for isolation of cryptographic operations. Note: Erratum E16 (voltage glitch bypass of security config reads) is a known residual risk in current silicon; see Raspberry Pi RP2350 errata documentation.
5. **True 2FA:** TOTP (RFC 6238) generation on a 32-bit platform with RTC and secure secret storage.

Firmware Snippets

A.1 Tactile Switch — Main Loop

The main loop polls PB1 for a LOW signal (button active-low). A 20 ms debounce delay is followed by a second read to confirm the press before injecting the keystroke sequence.

```
1 // Tactile switch variant active-low, internal pull-up enabled
2 while (1) {
3     usbPoll();
4     if (!(PINB & (1 << BUTTON_PIN))) { // PB1 LOW = pressed
5         _delay_ms(20); // debounce
6         if (!(PINB & (1 << BUTTON_PIN))) {
7             type_password();
8             while (!(PINB & (1 << BUTTON_PIN))) {
9                 usbPoll(); // wait for
10                    release
11            }
12        }
13    }
```

A.2 Touch Pad (TTP223) — Main Loop

The TTP223-BA6 drives PB1 HIGH on touch. No internal pull-up is enabled. The loop waits for PB1 to go LOW (touch released) before re-arming, preventing repeated triggers from a sustained touch.

```
1 // Touch pad variant active-high, NO internal pull-up on PB1
2 // DDRB &= ~(1 << BUTTON_PIN); // input (already default)
3 // Do NOT set: PORTB |= (1 << BUTTON_PIN); no pull-up for TTP223
4 while (1) {
5     usbPoll();
6     if (PINB & (1 << BUTTON_PIN)) { // PB1 HIGH = touched
7         _delay_ms(20);
8         if (PINB & (1 << BUTTON_PIN)) {
9             type_password();
10            while (PINB & (1 << BUTTON_PIN)) {
11                usbPoll(); // wait for
12                    release
13            }
14        }
15    }
```

A.3 Password Suffix Storage

The suffix is a C string literal compiled directly into Flash. It occupies the exact number of ASCII bytes plus a null terminator. EEPROM is not used; the suffix is erased and replaced each time firmware is reflashed.

```
1 // Stored in Flash (program memory), NOT in EEPROM.
2 // Change this string and reflash to update the suffix.
```

```

3 static const char password_suffix[] = "!!#2025_complex";
4 // sizeof(password_suffix) = 15 (14 chars + null terminator)
5 // Flash consumption: 14 bytes of data + code to iterate it.

```

A.4 V-USB Callback Function

V-USB calls `usbFunctionSetup()` for every incoming USB control request. The only request handled here is the host's `GET_DESCRIPTOR` for the HID Report Descriptor.

```

1 USB_PUBLIC uchar usbFunctionSetup(uchar data[8]) {
2     usbRequest_t *rq = (void *)data;
3
4     if ((rq->bmRequestType & USBRQ_TYPE_MASK) ==
5         USBRQ_TYPE_STANDARD) {
6         if (rq->bRequest == USBRQ_GET_DESCRIPTOR) {
7             if (rq->wValue.bytes[1] ==
8                 USBDESCR_HID_REPORT) {
9                 usbMsgPtr = (void *)
10                    usbHidReportDescriptor;
11                 return sizeof(usbHidReportDescriptor)
12                    ;
13             }
14         }
15     }
16     return 0; // All other requests unhandled
17 }

```

A.5 Fuse Programming Commands (Reference)

```

1 # Set fuses: 16 MHz external crystal + BOD at 2.7V (run ONCE per chip)
2 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
3 -U lfuse:w:0xFF:m -U hfuse:w:0xDD:m
4
5 # Verify fuses were written correctly
6 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
7 -U lfuse:v:0xFF:m -U hfuse:v:0xDD:m
8
9 # Revert to internal 8 MHz oscillator (crystal no longer needed)
10 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
11 -U lfuse:w:0xE2:m -U hfuse:w:0xDF:m
12
13 # Flash new firmware
14 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
15 -U flash:w:main.hex
16
17 # Read Flash back (extract stored content security check)
18 avrdude -c avrisp -p attiny85 -P /dev/ttyACM0 -b 19200 \
19 -U flash:r:dump.hex:i

```